SIMOG System Architecture: A Technical Deep Dive for Architects and Developers

Introduction: System Purpose and Architectural Goals

This document provides a comprehensive architectural and infrastructural overview of the SIMOG system. It is intended to serve as the primary technical reference for architects and software developers responsible for the system's maintenance, internalization, and domain-specific adaptation. By detailing the technology stack, modular design, runtime behavior, and operational procedures, this report provides a definitive blueprint that connects the system's strategic technological choices to its day-to-day operational realities, enabling informed architectural decisions.

The core purpose of the SIMOG system is to manage the lifecycle of public tenders and lots, providing a web-based interface for operational management, SOAP-based endpoints for system-to-system integration, and command-line utilities for bulk data processing. The architecture is designed to support complex business workflows, secure authentication, internationalization, and robust auditing.

This document is structured to provide a complete technical blueprint of the system. It begins by examining the core technology stack and design philosophy, followed by a detailed breakdown of the modular architecture. Subsequent sections deconstruct the application's runtime and execution model, the data persistence layer, and the centralized configuration system. Finally, the report covers security, identity management, internationalization, and provides practical guidance on build, deployment, and operational procedures, concluding with an analysis of key architectural patterns to guide future development.

Core Technology Stack and Design Philosophy

Understanding the foundational technology stack is of strategic importance, as the specific choice of technologies and their versions dictates the system's capabilities, performance characteristics, and long-term maintenance requirements. The SIMOG platform is built on a mature, enterprise-grade Java stack, prioritizing stability and established patterns over cutting-edge frameworks.

The following table details the primary technologies that constitute the SIMOG ecosystem.

Core Technology Stack

Component	Specification & Version
Programming Language	`Java 6 (Source/Target 1.6)`
Application Server	`JBoss AS 5.x` (Servlet 2.4, JSP 2.0). Context files for `Tomcat 5` are also available for development environments.
Build System	`Apache Maven` (Multi-module project)
Database	Microsoft SQL Server
SOAP Webservices	`Apache Axis 1.4` (in `SimogWeb`) and `JBossWS Native` (JAX-WS) for dedicated service endpoints. `XMLBeans` for data binding.
Frontend Technologies	JavaServer Pages (`JSP 2.0`) with `JSTL`, Custom Tags, Servlets, JavaScript (`jQuery`, `jQuery UI`)
Logging Framework	`Log4j 1.2.13` with MDC (Message-Driven Context) enrichment.

The selection of `Java 6` and `JBoss AS 5.x` firmly positions this as a legacy stack. For developers and architects, this implies a development environment constrained to features available in older Java versions and Servlet specifications. While this ensures stability and predictability, it also presents potential challenges for modernization, integration with contemporary libraries, and sourcing

development talent. Any future evolution of the system must carefully consider a migration path for these core components.
Having established the foundational technologies, we now turn to the specific modular structure of the application, which organizes the codebase into logical, reusable components.

Modular Architecture: A Component-Based Breakdown

The SIMOG system is architected as a multi-module Maven project, a design choice that promotes a clean separation of concerns, enhances code reusability, and streamlines the build process. Each module has a distinct technical role, encapsulating a specific layer of functionality, from frontend presentation to backend data access and external service integration. This component-based structure is fundamental to the system's maintainability.
The table below provides a detailed breakdown of each Maven module, its packaging format, and its specific technical role within the overall architecture.

SIMOG Maven Modules and Technical Roles

Module	Packaging	Technical Role and Key Dependencies
`SimogWeb`	`war` (AVCP-SimogWeb)	The main web application. Implements an MVC pattern with Servlets and JSPs, hosts legacy Axis SOAP services, and includes UI filters. Depends on `SimogCommon`, `SimogWSCommon`, `SimogAVCPLogin`, `XmlAuth`.
`SimogCommon`	`jar`	Contains shared utilities, the custom JDBC DAO layer, configuration properties loader (`SimogProperties`), domain constants, and XMLBeans integration logic.
`SimogWSCommon`	`jar`	Provides common SOAP clients (Axis, HttpClient), the shared configuration manager (`ConfigurationManager`), and PDD XMLBeans bindings.
`SimogAVCPLogin`	`jar`	Manages authentication integration with AVCP/IAM via SAML 2.0 (using `opensaml 2`). Depends on `SimogCommon`.
`SimogFlusso`	`jar`	The application's workflow engine, containing `WorkFlow*` classes for tenders, lots, and adhesions. Reused by both web and batch modules.
`SimogMassLoader`	`jar` (CLI)	A command-line utility for massive XML imports from the file system, handling validation and feedback generation. Uses `SimogWSCommon` and `SimogFlusso`.
`SimogWSAGG`	`war`	A dedicated SOAP endpoint for aggregator entities (`/services/SimogWSAGG`), also exposing standard PDD services.
`SimogWSPDD`	`war`	The primary PDD SOAP endpoint, exposing services like `SimogWSPDD`, `LoaderAppaltoWS`, and others.
`SimogTEDCommon`	`jar`	Encapsulates integration with TED, utilizing JPA (Hibernate) via the `ted` persistence-unit, a Jersey REST client, and JSON handling.
`SimogWSTED`	`war`	Exposes the `/services/SimogWSTED` SOAP endpoint for the TED workflow. Depends on `SimogTEDCommon` and `TEDXSD`.
`SimogXmlBeans`	`jar`	Contains XMLBeans bindings generated from the core domain XSDs (tenders, lots, forms).
`SimogXmlBeansPDD`	`jar`	Contains specific PDD XMLBeans bindings, pre-built using an Ant script. Includes the pre-built artifact `SimogXmlBeansPDD-3.04.7.jar`.
`XSD`	Source Files	Contains the source XML Schemas for the application domain, used for data validation and code generation.

This modular design yields significant architectural benefits. By separating the main web application (`SimogWeb`) from shared utilities (`SimogCommon`), workflow logic (`SimogFlusso`), and distinct SOAP endpoints (`SimogWSPDD`, `SimogWSAGG`), the system achieves high cohesion and low coupling. This

structure makes the application easier to understand, test, and extend, as changes in one functional area are less likely to impact others.
This static, compile-time structure defines the system's components. The next section will explore how these modules interact and execute at runtime.

Application Runtime and Execution Model

This section deconstructs the runtime behavior of the SIMOG application, moving from the static code structure to its dynamic execution. It covers the lifecycle of a web request in the primary web application, the function of background processes, and the specifics of the exposed service endpoints.

SimogWeb: The Primary Web Application

`SimogWeb` is the core user-facing component of the system. Its runtime behavior is governed by a classic server-side MVC pattern implemented with Servlets and JSPs.
The HTTP request lifecycle follows a well-defined sequence orchestrated by servlet filters:

- **LocaleFilter**: This is the first filter in the chain. It enforces UTF-8 character encoding for all requests and responses. It then determines the user's locale by checking for a locale request parameter, a value in the user's session, or the Accept-Language header, ensuring proper internationalization.
- **AuthenticationFilter**: This filter manages security and auditing. It enriches the Log4j Message-Driven Context (MDC) with user session data (session ID, login, and contracting authority CF) for detailed logging. When active, it integrates with the IAM/SAML infrastructure to handle authentication.

The controller layer is composed of numerous dedicated servlets, each responsible for a specific function, such as managing tenders (`SrvInserisciGara`), lots (`SrvGestisciLotto`), or forms (`SrvSchedaA`). The user session is configured with a default timeout of 60 minutes.
The frontend is rendered using JSP 2.0 with the JSTL tag library. Custom tags, such as `utils:message`, are used to retrieve localized text from resource bundles. Client-side interactivity is powered by JavaScript using the jQuery and jQuery UI libraries, with explicit support for Right-to-Left (RTL) languages.

Service Endpoints (WARs)

In addition to the main web application, SIMOG deploys several distinct SOAP service endpoints as separate WAR files. This isolates service traffic and allows for independent deployment and scaling. The key endpoints include:

- **SimogWSPDD**: Exposes core PDD services at paths like /services/SimogWSPDD, /services/LoaderAppaltoWS, /services/SimogWSOSS, and /services/LoaderAppaltoAVCPASS.
- **SimogWSAGG**: Provides services for aggregator entities at /services/SimogWSAGG.
- **SimogWSTED**: Exposes a dedicated service for the TED workflow at /services/SimogWSTED.

Batch Processing and CLI Tools

For offline and bulk data processing, the system provides the `SimogMassLoader` command-line utility. Its purpose is to perform massive imports of XML files from the local file system. The utility operates on a set of predefined directories (`IN`, `WRK`, `OUT`, `BKP`) located relative to a base path that is passed as a command-line argument. This structure allows for flexible configuration of the processing pipeline at runtime.

Data Persistence and Schema

The SIMOG system employs a hybrid approach to data persistence, combining a custom-built JDBC Data Access Object (DAO) layer for core application logic with the Java Persistence API (JPA) for specific integrations. This design reflects an evolution over time, with the custom DAO layer forming the foundation and JPA being introduced for newer modules.

Data Access Layers

- **Custom JDBC DAO Layer**: The primary data access mechanism resides in the SimogCommon module. It follows a consistent pattern where AccessiDB classes encapsulate database connections, and dedicated DAO "Manager" classes (e.g., VarianteManager) execute prepared statements. SQL queries are constructed using constants from classes generated directly from the database schema, ensuring type safety and reducing runtime errors.
- **JPA Implementation**: For the TED integration, the SimogTEDCommon and SimogWSTED modules utilize JPA. The configuration is defined in persistence.xml with a persistence-unit named ted. It leverages Hibernate as the persistence provider and is configured for schema validation.

Database Connectivity

The application connects to its Microsoft SQL Server database through a JNDI resource named `java:jdbc/SIMOG`. This decouples the application from the specific database connection details, which are managed by the application server.
For JBoss, the JNDI binding is configured in the `jboss-web.xml` file located within each WAR. For Tomcat (used in development environments), the connection pool is configured in environment-specific template files (e.g., `context.xml.local`, `context.xml.demo`). These files contain crucial performance-tuning parameters for the connection pool, such as `maxActive` and `maxIdle`.

Key Data Model Structures

The following are excerpts from the SQL schema, representing key tables in the SIMOG data model.

- **WS_SESSIONS**: Manages web service sessions, storing user details, tickets, and session status.
- **VARIANTI**: Stores information about variations to a tender, linking them to an adjudication and a status.
- **WS_KEYS**: Stores consultation keys for web service sessions, managed by WSSessionManager.
- **STATI_SCHEDA**: A reference table that defines the possible states in the application's workflows.

Centralized Configuration Management (simog.ini)

The `simog.ini` file serves as the central nervous system for the SIMOG application. Its critical role is to externalize all environment-specific settings, integration endpoints, and feature toggles, allowing administrators to modify system behavior without requiring code changes or redeployments. This approach is fundamental to the system's flexibility and operational manageability across different environments (development, testing, production).
The configuration is loaded at startup by the `ConfigurationManager` and `SimogProperties` classes. The loading mechanism first checks for the file at the default production path (`/opt/SIMOG/simog.ini`) and, if not found, falls back to a local development path (`C:\\home\\SIMOG\\simog.ini`).
The keys within `simog.ini` can be grouped into several major functional categories:

- **Database Connectivity and Logging**: Examples: JDBC_DATASOURCE_NAME, LOG_CONFIG_FILENAME.

- **File System Paths**: Examples: APP_ROOT_DIR, PATH_ALLEGATI, SIMOG_TO_ESATTORE_DIR.
- **Security, SSO, and IAM**: Examples: SAML_LOGIN_URL, SAML_CERTIFICATE, WS_AUTH_TARGET_HOST, trustStore.
- **External Service Integrations**: Examples: WSANAG_URL, CIG_PCP_URL, RGS_URL, WS_AVCPASS_HOST.
- **Domain Logic and Feature Toggles**: Examples: DATA_ATTIVAZIONE_BIM (date-based feature activation), ATTIVAZIONE_BLOCCO_CIG (a boolean-like feature flag), IMPORTO_MIN_LOTTO (a business rule parameter).

Security, Identity, and Internationalization

This section focuses on the critical non-functional requirements of security, authentication, authorization, and internationalization.

Authentication and Identity

- **Primary Authentication (SAML)**: The default authentication mechanism integrates with AVCP IAM (Identity and Access Management) using the SAML 2.0 protocol, powered by the opensaml 2 library.
- **Alternate "Local" Authentication**: For development or testing, setting WS_AUTH_TARGET_HOST=LOCAL in simog.ini bypasses the SAML flow.

Authorization and Auditing

- **Authorization**: The system utilizes the Java Authentication and Authorization Service (JACC) API for container-managed authorization.
- **Auditing**: The AuthenticationFilter enriches the log4j MDC with a composite key of sessionId_login_CFStazioneAppaltante, ensuring every log entry can be traced back to a specific user session.

Data Security

- **Secure Channels**: The application uses a keystore (SimogWeb/SimogTrust/.keystore) and a configurable truststore (defined in simog.ini) for secure communications.
- **Antivirus Scanning**: Optional integration with ClamAV antivirus engine is available, configured via CLAMAV_HOST and CLAMAV_PORT in simog.ini.

Internationalization (i18n) and Localization (L10n)

- **i18n Architecture**: The LocaleFilter sets UTF-8 encoding and determines the locale. All user-facing text is externalized into ResourceBundle files (e.g., messages_it.properties). The utils:message custom tag is used in JSPs.
- **RTL Support**: A dedicated CSS file (rtl.css) provides support for Right-to-Left languages.

Build, Deployment, and Operations

This section outlines the process for building, deploying, and maintaining the SIMOG system.

Build Process

The system is built using Apache Maven.

- **Build Commands**: A full build is done with mvn clean install. A single module can be built with mvn -pl SimogWeb -am package.

- **Primary Artifacts**: The build generates deployable WAR files (AVCP-SimogWeb.war, SimogWSPDD.war, etc.) and shared JAR files.

Deployment Procedure

1. Configure the java:jdbc/SIMOG datasource on the application server.
2. Place simog.ini and simog_log4j.properties in the designated directory (e.g., /opt/SIMOG/).
3. Ensure the application server user has read/write permissions on file paths defined in simog.ini.
4. Deploy the generated WAR files to the application server.

Operational Maintenance

- **Hot-Reloading Configuration**: The /reloadProperties endpoint (SrvReload servlet) allows for live reloading of the simog.ini file without a server restart.
- **Application Versioning**: The build version is tracked in a simogversion.properties file and is accessible at runtime via ConfigurationManager.buildVersion.

Architectural Patterns and Extension Guidelines

This section synthesizes key architectural patterns to guide future development.

Prominent Design Patterns:

- **DAO/Manager with Generated Constants**: Data access layer uses Manager classes with SQL queries referencing generated constants for type safety.
- **Factory and Strategy for Workflows**: WorkFlowFactory acts as a Factory Method to instantiate different WorkFlow* Strategy implementations.
- **Configuration Singleton**: ConfigurationManager implements a thread-safe singleton to provide global access to properties from simog.ini.
- **Template Method for Logging**: LogBloccoDatiManager provides a template for standardized database-level logging of CRUD operations.

Guidelines for Domain Extensions

- **Prioritize Configuration over Code**: Use feature flags and parameters in simog.ini (e.g., DATA_ATTIVAZIONE_*) before hardcoding new logic.
- **Managing XML Schemas**: Update XSD files first, then regenerate XMLBeans data binding classes.
- **Customizing Authentication**: Set WS_AUTH_TARGET_HOST=LOCAL in simog.ini to bypass SAML and implement custom authentication in AuthenticationFilter.
- **UI and I18n Best Practices**: Add new text to messages_*.properties files and use the utils:message tag in JSPs.
- **Performance Tuning**: Calibrate datasource connection pool settings (maxActive, maxIdle) and session timeouts for the target environment.

Introduction

As global markets become increasingly interconnected, the ability for software applications to support multiple languages and cultural conventions is no longer a luxury but a necessity. This document provides a comprehensive technical overview of the internationalization (**i18n**) and localization (**l10n**) architecture implemented in the **SimogWeb** platform. Specifically, it details the mechanisms for handling multiple languages, including the complexities of right-to-left (**RTL**) language support for languages such as Arabic. The goal is to create a seamless and intuitive user experience for a diverse, global user base. This involves a multi-layered strategy encompassing the user interface (UI), backend business logic, and data persistence layers.

The Core Mechanism: Spring Framework's Locale Support

The foundation of **SimogWeb**'s internationalization strategy is built upon the robust and flexible i18n capabilities of the Spring Framework. Spring provides a comprehensive set of tools for resolving user locales and managing language-specific resources. The core components leveraged in our architecture are the **LocaleResolver** and the **MessageSource**.
In our implementation, we use a combination of **SessionLocaleResolver** and **LocaleChangeInterceptor**.

- The **SessionLocaleResolver** stores the selected locale in the user's HTTP session. This ensures that the user's language preference is maintained throughout their session, providing a consistent experience as they navigate across different pages of the application.
- The **LocaleChangeInterceptor** intercepts incoming web requests and checks for a specific URL parameter (e.g., ?locale=ar). If this parameter is present, the interceptor updates the locale in the **SessionLocaleResolver**, effectively changing the language for the current and subsequent requests within that session. This provides a straightforward and stateless way for users to switch languages via simple links.

Implementation Details

UI Language Switch

The user-facing language selection is implemented as a simple dropdown or a set of links in the application's header. The process flow is as follows:

1. A user clicks on a language option, for instance, "العربية" (Arabic).
2. This action triggers a request to the current page URL but appends the locale parameter, such as ?locale=ar.
3. The **LocaleChangeInterceptor** on the backend detects this parameter and updates the user's session locale to Arabic.
4. The page is then re-rendered. Spring's view layer automatically uses the new locale to fetch the appropriate translated strings from the resource bundles.

Resource Bundles

All user-facing text, such as labels, button text, and messages, is externalized into property files known as resource bundles. This practice decouples the text from the application code, making it easy for translators to work on language files without needing to touch the source code.

- The base resource bundle is named **messages.properties** and contains the default language strings (English).

- For each supported language, a corresponding file is created with a language-specific suffix. For example:
  - **messages_ar.properties** for Arabic.
  - **messages_fr.properties** for French.

These files are typically located in the `WEB-INF/classes/` directory of the deployed application, which corresponds to the project's classpath. The Spring Boot configuration for managing these bundles is straightforward, requiring only a few properties in the main configuration file:

```
# Specifies the base name of the resource bundles
spring.messages.basename=messages

# Ensures properties files are read using UTF-8 encoding
spring.messages.encoding=UTF-8
```

The `spring.messages.encoding=UTF-8` property is critically important. It ensures that files containing non-Latin characters, such as Arabic or Cyrillic, are read and processed correctly, preventing character encoding issues that lead to garbled text (mojibake).

Backend Logic

In the backend Java code, the **ResourceBundleMessageSource** bean is injected wherever translated messages are needed (e.g., for generating email notifications, API error responses, or log messages). By accessing the current request's locale, the backend can retrieve the correct translation dynamically, ensuring that all system-generated communication adheres to the user's selected language.

Database Unicode Strategy

Properly storing and retrieving multilingual data from the database is a critical pillar of a successful i18n strategy. The **SimogWeb** application is configured to handle **Unicode** data end-to-end, from the database schema to the application layer. Our primary database is Oracle, and the strategy involves three key areas.

Database Column Configuration

To support a wide range of characters from different languages, the database character set is configured to **AL32UTF8**, which is Oracle's recommended implementation of the UTF-8 standard. Furthermore, specific column types are chosen based on their ability to store Unicode characters efficiently.

Column Type	Storage Method
NVARCHAR2	Used for columns that will exclusively store multilingual text. It stores data in a national character set, which is ideal for Unicode.
VARCHAR2	Can be used if the database character set is **AL32UTF8**. However, **NVARCHAR2** is often preferred for clarity and to avoid potential misconfigurations.
NCLOB	Used for large text objects (e.g., user-generated content, detailed descriptions) that require Unicode support. It is the national character set equivalent of a **CLOB**.

JDBC Connection Parameters

The connection string used by the application's data source must be configured to handle Unicode correctly. For JDBC connections, this is achieved by adding specific parameters to the connection URL. A typical configuration looks like this:

```
?useUnicode=true&characterEncoding=UTF-8
```

These parameters instruct the JDBC driver to expect and handle character data using UTF-8 encoding, preventing data corruption during transit between the application and the database server.

Data Handling in Java

Within the Java application, all string manipulations are inherently Unicode-aware, as Java's `String` class is based on UTF-16. However, when interacting with the database, especially with Oracle's national character set columns (**NVARCHAR2**, **NCLOB**), it is best practice to use specific methods to ensure correct data handling. We use a utility class, **UnicodeHelper**, which contains methods like `setStringSmart`. This method correctly uses `preparedStatement.setNString()` for national character set columns, ensuring that the JDBC driver handles the data appropriately.

```
UnicodeHelper.setStringSmart(preparedStatement, index, value);
```

RTL (Right-to-Left) Support

Supporting **RTL** languages like Arabic and Hebrew goes beyond simple text translation. It requires a fundamental change in the layout and orientation of the user interface. Our approach combines CSS and dynamic server-side logic to achieve a complete and correct **RTL** experience.

CSS Strategy for RTL

The core of our **RTL** strategy is a CSS-based approach. We maintain a main stylesheet for the default **LTR** (Left-to-Right) layout. A separate, override stylesheet is created specifically for **RTL** adjustments.

- **direction.css**: Contains the default **LTR** styles (e.g., float: left;, margin-left: 10px;, text-align: left;).
- **direction.rtl.css**: Contains overrides for **RTL** layouts. This file is loaded only when an **RTL** language is active. It uses attribute selectors to apply styles.

The **RTL** stylesheet uses a root-level attribute selector, `html[dir="rtl"]`, to scope all its rules. This ensures that the **RTL** styles are only applied when the `dir` attribute of the tag is set to `"rtl"`.

```
/* Example from direction.rtl.css */
html[dir="rtl"] .some-element {
    float: right; /* Flipped from left */
    margin-left: 0;
    margin-right: 10px; /* Flipped from margin-left */
    text-align: right; /* Flipped from left */
}
```

Modern CSS logical properties (e.g., `margin-inline-start`, `padding-inline-end`) are being progressively adopted to simplify this process, as they automatically adjust to the document's writing direction. However, the override stylesheet approach provides maximum browser compatibility and explicit control.

Dynamic Direction Switching

The application's main view template (e.g., a master JSP or Thymeleaf layout) is responsible for setting the document's direction. It dynamically inspects the current locale resolved by Spring.

If the language code is 'ar' (Arabic), 'he' (Hebrew), or any other configured **RTL** language, the template injects `dir="rtl"` into the tag and includes a link to the **direction.rtl.css** stylesheet. It may also add an **RTL**-specific class to the tag (e.g.,) for additional styling hooks. Otherwise, it defaults to `dir="ltr"` and omits the **RTL** stylesheet. This server-side logic ensures that the correct layout is rendered from the very first paint, avoiding any flash of unstyled or incorrectly styled content.

The internationalization and **RTL** support in **SimogWeb** is a comprehensive, multi-layered solution that addresses the challenges of building a truly global application. By leveraging the power of the Spring Framework for locale management, externalizing all user-facing strings into resource bundles, ensuring end-to-end **Unicode** support at the database level, and implementing a robust CSS strategy for **RTL** layouts, the platform provides a culturally appropriate and user-friendly experience for all users, regardless of their language or region. This robust architecture ensures that **SimogWeb** is not only capable of serving its current user base but is also well-prepared for future expansion into new languages and markets.